

DOSSIER DE CONCEPTION

Document de travail — à valider avant développement

ÉTAPE 6

Identité et connexion par QR code

Une seule brique de scan, deux usages

Ajout de contact instantané — connexion multi-appareils



Identité



Connexion

Périmètre Talky (Flutter) ■ Alanya-Backend

Statut Implémenté — relu et corrigé

Dépend de Rien — indépendant des cinq premiers volets

Migration 026_qr_appareils.sql

Date 30 juillet 2026

Avant-propos

Deux fonctionnalités demandées, toutes deux fondées sur le scan d'un QR code : un code d'identité personnel qui ajoute instantanément un contact préféré, et une connexion multi-appareils façon « appareil de confiance qui en autorise un nouveau ». Elles sont traitées dans un seul dossier parce qu'elles partagent une vraie infrastructure — le composant scanner, le générateur de QR stylé, l'enveloppe de résolution d'un code scanné — au même titre que compte officiel et diffusion l'ont été à l'étape 3.

Comment lire ce document. La **partie I** décrit la fonctionnalité telle que l'utilisateur et le client la voient. La **partie II** décrit comment on la construit. On peut valider la partie I sans lire la partie II.

Table des matières

Avant-propos	1
I Approche naïve	4
1 Le besoin	5
1.1 Ce qui existe déjà	5
1.2 Ce qu'on ajoute	5
1.3 Les règles produit	6
2 Mon code personnel	7
2.1 Vue d'ensemble	7
2.2 Écran — Mon code	8
2.3 Être prévenu, ajouter en retour	9
2.4 Reconnaître un contact ajouté par QR	9
2.5 Partager son code	10
2.6 Écran — Scanner	10
2.7 Points d'entrée	10
3 Connexion d'un second appareil	11
3.1 Le principe	11
3.2 Écran — Se connecter par QR	11
3.3 Écran — Confirmer la connexion	13
3.4 Écran — Mes appareils connectés	13
3.5 Points d'entrée	15
II Approche technique	16
4 Enveloppe de payload et résolution	17
5 Modèle de données	18
5.1 Le jeton d'identité permanent, et son verrou	18
5.2 La nature de <code>device_id</code>	18
5.3 Le jeton et la table <code>appareils</code>	19
5.4 L'origine d'un contact	20
6 Rendre la révocation réelle	22

6.1	Ce que fait vraiment la révocation	23
7	Ajout de contact instantané	24
7.1	Le jeton éphémère, en mémoire	24
7.2	Résolution et consommation	25
7.3	Prévenir le propriétaire	26
7.4	L'ajout en retour	26
8	La session de connexion éphémère	27
8.1	Une structure en mémoire, pas une table	27
8.2	Deux secrets par session, jamais interchangeables	27
8.3	Cycle de vie	28
8.4	L'exception d'authentification socket	29
9	API	30
9.1	Limitation de débit	31
10	Client Flutter	32
10.1	Couche données	32
10.2	Interface	34
10.3	Mon code : décompte, consommation, régénération	35
10.4	La pastille, rendue à un seul endroit	35
10.5	Le dialogue global d'ajout en retour	35
10.6	Liens profonds	36
11	Impact fichier par fichier	37
11.1	Backend — Alanya-Backend	38
11.2	Mobile — Talky (Flutter)	40
12	Vérification	42
12.1	Backend	42
12.2	Client	42
13	Points de vigilance et questions ouvertes	44

PARTIE I

Approche naïve

CHAPITRE 1

Le besoin

1.1 Ce qui existe déjà

Alanya n’a aujourd’hui **qu’un seul chemin** pour ajouter un contact préféré : la recherche manuelle par nom, pseudo ou Alanya ID, depuis la feuille d’ajout du profil. Aucun lien d’invitation, aucun code, aucun geste plus rapide.

Vérifié dans le code. `lib/widgets/add_contact_sheet.dart` interroge d’abord le cache local (`filterUsersBySearch`), puis `GET /users/search` après 400 ms sans résultat. L’ajout appelle `POST /contacts/:userId` — la même route que celle que ce volet réutilisera pour l’ajout par QR.

Côté connexion, Alanya n’a **qu’un seul appareil implicite** : se connecter sur un second téléphone fonctionne déjà techniquement (rien ne l’empêche), mais rien ne le montre non plus. Il n’existe aucun écran « mes appareils », aucun moyen de savoir où son compte est ouvert, ni de fermer une session à distance.

Vérifié dans le code. `lib/screens/profile/account_security_screen.dart` ne contient que deux sections : *Email* et *Changer le mot de passe*. Rien sur les appareils.

1.2 Ce qu’on ajoute

Deux fonctionnalités, toutes deux fondées sur un QR code :

1. **Un code personnel éphémère.** Chaque utilisateur génère, en ouvrant « Mon code », un QR à son identité — logo Alanya au centre, traits arrondis, une vraie carte de visite plutôt qu’un carré noir et blanc. Le scanner d’un autre utilisateur reconnaît la personne et l’ajoute *instantanément* en contact préféré. Le code vit **dix minutes** et s’éteint au premier scan réussi ; l’écran en régénère un neuf tout seul, et le propriétaire est prévenu — et invité à ajouter l’autre **en retour** — chaque fois que son code est utilisé.
2. **La connexion d’un second appareil.** Un nouvel appareil affiche un code ; un appareil déjà connecté le scanne pour l’autoriser. L’utilisateur peut ensuite voir, depuis son compte, la liste de tous les appareils connectés — et en déconnecter un à distance.

Une seule brique, deux usages. Les deux fonctionnalités ne partagent pas qu’un mot dans leur nom. Elles partagent un **composant scanner**, un **générateur de QR stylé**, et une manière commune de résoudre ce qu’un code scanné signifie. C’est pour cette raison qu’elles sont conçues dans un seul volet — le même raisonnement qui a réuni compte officiel et diffusion dans l’étape 3.

1.3 Les règles produit

Qui peut scanner	Un utilisateur déjà connecté , dans les deux cas. Scanner n'est jamais un geste public ; c'est toujours un compte qui en reconnaît un autre.
Vie du code personnel	Dix minutes, une seule personne. Généré à la demande à l'ouverture de « Mon code », consommé au premier scan réussi. L'écran affiche un compte à rebours et régénère automatiquement — à l'expiration comme après consommation.
Ajout de contact	Instantané , sans écran de confirmation. Le filet de sécurité est une carte de résultat annulable, pas une étape supplémentaire.
Retour au propriétaire	Prévenu à chaque scan de son code, où qu'il soit dans l'application, et invité à ajouter l'autre en retour — notification si l'app est en arrière-plan, simple information si le lien existait déjà.
Contacts ajoutés par QR	Reconnaissables partout : pastille sur l'avatar, mention datée sur la fiche, filtre « Par QR » dans les contacts préférés <i>et</i> dans la liste des discussions.
Note de contexte	Optionnelle, saisie juste après le scan — « rencontré au salon de Douala » — ou au moment d'ajouter en retour, et relue sur la fiche du contact. Elle appartient à la relation : l'autre ne la voit jamais.
Codes permanents	Réservés aux futurs comptes business (volet 1). Aucun compte personnel n'a de code qui dure.
Connexion d'un appareil	Jamais instantanée. L'appareil qui scanne voit toujours l'identité de l'appareil qu'il autorise, et doit confirmer explicitement.
Durée du QR de connexion	Environ 90 secondes , puis régénération automatique — comme un code à usage unique, pas comme une carte de visite.
Emplacements	Profil (mon code), feuille d'ajout de contact (scanner), Compte et sécurité (appareils connectés), écran de connexion (nouvel appareil).

Pourquoi un code qui meurt. Un code permanent est une affiche ; un code de dix minutes est une poignée de main. Le code personnel sert à des rencontres — on le montre, on l'envoie, la personne en face scanne, c'est fini. L'usage unique garantit que le code n'ajoute que la personne à qui on vient de le tendre : une capture d'écran qui circule ensuite ne mène plus nulle part. Et comme l'écran régénère aussitôt, montrer son code à trois personnes d'affilée reste trois gestes de *leur* côté, zéro du sien.

Vérifié dans le code — pourquoi le permanent est verrouillé. L'infrastructure du code permanent (`users.qr_public_id`) existe et reste en place, mais son garde refuse aujourd'hui tout le monde — et pour une raison précise : la colonne `users.type_compte`, qu'on prendrait volontiers pour un type de compte métier, est en réalité un **rôle d'administration** (0 utilisateur, 1 admin, 2 super-admin, cf. la route `PUT /users/:id/role` de `src/routes/admin.js`, côté Alanya-Backend). Rien dans la base ne sait encore dire « ce compte est un commerce ». Le volet 1 (socle de compte) apportera `account_type` ; le verrou des codes permanents s'y branchera alors.

CHAPITRE 2

Mon code personnel

2.1 Vue d'ensemble

Un seul écran, deux volets accessibles d'un geste : *Mon code* et *Scanner* — comme on bascule entre appareil photo avant et arrière.

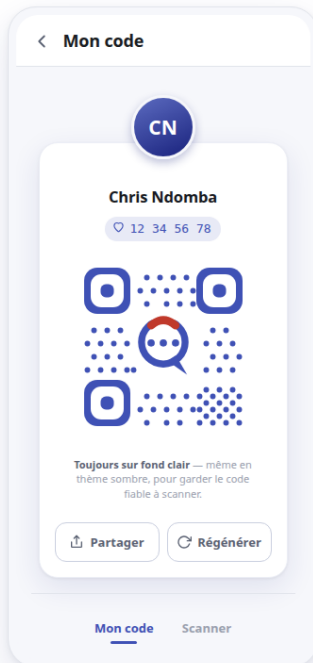
ALANYA · IDENTITÉ ET CONNEXION PAR QR CODE · ÉTAPE 6

Un code, une identité — pas un carré noir et blanc

Le code de chaque utilisateur porte le monogramme Alanya en son centre, des yeux et des modules arrondis. Le scanner reconnaît la personne et l'ajoute en contact préféré instantanément — sans écran de confirmation intermédiaire.

Les deux écrans

Un seul écran, deux volets accessibles par un geste : « Mon code » et « Scanner ».



MON CODE
MÉDAILLON, LOGO AU CENTRE,
PARTAGE ET RÉGÉNÉRATION



SCANNER
RECONNAISSANCE INSTANTANÉE,
SUCCÈS DOUX ET ANNULABLE

Ajout instantané, sans écran de confirmation. C'est la règle produit demandée — scanner suffit. Le filet de sécurité est le bandeau « Annuler » (5 s), pas un écran intermédiaire.

Le code reste lisible en toute condition. Correction d'erreur niveau H (30 % de redondance) pour tolérer le logo central sans nuire à la reconnaissance.

Régénérer invalide l'ancien code. Utile si un code a été capturé par erreur ou partagé publiquement — il ne mène alors plus à personne.

Tokens : lib/core/theme/app_colors.dart qr_flutter - eyeStyle/dataModuleStyle personnalisés, correction H mobile_scanner - scan plein écran
Maquette de conception - aucun code applicatif

Maquette interactive. claude.ai/code/artifact/69e9cbec

Source versionnée : docs/architecture/maquettes/qr-identite.html

2.2 Écran — Mon code

Une carte blanche : médaillon d'avatar débordant le haut, nom, Alanya ID, puis le QR code lui-même — fond clair, points arrondis à la couleur de marque, logo Alanya au centre. Sous le

code, un **compte à rebours** (« Expire dans 9 :41 ») et la mention « Valable 10 minutes et pour une seule personne ». En dessous, deux actions : **Partager** et **Nouveau code**.

À zéro — comme après un scan réussi — l'écran régénère un code neuf de lui-même : l'utilisateur n'a jamais un code mort sous les yeux, exactement le comportement déjà retenu pour le QR de connexion (chapitre 3). « Nouveau code » reste disponible pour qui veut couper court sans attendre ; aucun dialogue de confirmation, renouveler un code qui expire tout seul n'a rien d'irréversible.

Pourquoi le code reste toujours clair. Même en thème sombre, la carte du QR reste sur fond blanc. Un code sur fond sombre scanne moins bien — la fiabilité du scan prime sur la cohérence du thème, à cet endroit précisément.

2.3 Être prévenu, ajouter en retour

Quand quelqu'un utilise votre code — scan ou lien —, vous êtes prévenu immédiatement : un dialogue propose d'ajouter la personne **en retour** (« untel vous a ajouté à ses contacts préférés avec votre code QR. L'ajouter en retour ? », *Ajouter / Non merci*). Si l'application est en arrière-plan ou fermée, une notification prend le relais. Et si vous aviez déjà cette personne en contact, pas de question : une simple information.

Le dialogue n'habite aucun écran. Le code a pu être partagé par lien : à l'instant où il est utilisé, son propriétaire peut être n'importe où dans l'application — pas forcément sur l'écran « Mon code ». Le dialogue est donc global, affiché où que l'on soit.

L'écran « Mon code », lui, réagit à sa manière : le code venant d'être consommé, il en affiche un neuf aussitôt — le propriétaire peut enchaîner plusieurs personnes sans un geste.

2.4 Reconnaître un contact ajouté par QR

Un contact ajouté par QR reste reconnaissable après coup, partout où il apparaît :

1. une **pastille QR** au coin bas-gauche de l'avatar — le bas-droit reste réservé au point de présence en ligne, les deux cohabitent — dans **toutes** les listes : contacts préférés, feuille d'ajout, nouvelle discussion, sélection de membres, partage de contact... et jusqu'à la liste des discussions ;
2. une **mention datée** sur la fiche du contact : « Ajouté par QR code · 30 juil. 2026 » ;
3. un **filtre** « Tous / Par QR » sur l'écran des contacts préférés, chaque option affichant son compte.

L'ajout *en retour* est marqué de la même façon : les deux directions du lien portent la même origine.

Maquette interactive. claude.ai/code/artifact/0e095614
Source versionnée : [docs/architecture/maquettes/qr-distinction.html](https://docs.architecture/maquettes/qr-distinction.html)

La liste des discussions porte le même filtre « Par QR » que l'écran des contacts préférés : un chip parmi les filtres existants (Tous, Discussions, Groupes...), qui ne retient que les conversations avec un contact rencontré par QR.

Enfin, la carte de résultat du scan propose un champ facultatif : une **note de contexte**, quelques mots pour se souvenir de la rencontre. Elle se retrouve sur la fiche du contact, en italique sous la mention d'origine — et nulle part ailleurs : c'est un aide-mémoire personnel, jamais montré à l'intéressé.

2.5 Partager son code

Partager ouvre un choix explicite : le **lien** (cliquable, accompagné de l'Alanya ID) ou l'**image** (la carte telle qu'affichée, compte à rebours compris). Dans les deux cas la légende annonce la couleur — « Ce code est valable 10 minutes et pour une seule personne » — suivie de l'Alanya ID : le code meurt en dix minutes, l'Alanya ID reste le chemin permanent, les deux voyagent ensemble.

Pas d'enregistrement dans la galerie. Enregistrer la carte en image n'est volontairement pas proposé : une image morte au bout de dix minutes finirait en code cassé au fond d'une pellicule. L'option reviendra avec les codes permanents des comptes business — eux supportent l'impression et l'affichage.

2.6 Écran — Scanner

Plein écran, viseur au centre, façon appareil photo. Dès qu'un code d'identité est reconnu, une **carte de résultat** se pose en bas de l'écran : la personne ajoutée (avatar, nom), « Ajouté à vos contacts », et trois suites possibles — **Message**, **Voir détails**, **Annuler**. La caméra reste vivante derrière la carte : on peut enchaîner plusieurs personnes sans ressortir de l'écran. Pas d'écran intermédiaire, pas de confirmation à taper.

Trois cas particuliers, traités comme des succès doux plutôt que des erreurs : scanner son propre code (rien ne se passe, un message discret l'indique — et le code *survit*), scanner un contact déjà favori (la carte s'affiche quand même, sans bouton Annuler : rien n'a été écrit), scanner un code expiré, déjà utilisé ou illisible (message clair, la caméra reste active).

Un bouton **Importer une image** couvre enfin le code reçu en capture d'écran ou en photo : l'image choisie dans la galerie est analysée exactement comme le cadre de la caméra — même résultat, même carte.

2.7 Points d'entrée

1. Le profil, une icône QR à côté de l'Alanya ID (`lib/screens/profile/profile_screen.dart`, en-tête).
2. La feuille d'ajout de contact, un bouton « Scanner un code » au-dessus de la recherche existante (`lib/widgets/add_contact_sheet.dart`).

CHAPITRE 3

Connexion d'un second appareil

3.1 Le principe

Contrairement à l'ajout de contact, une connexion ne s'approuve **jamais** au simple scan. Le nouvel appareil affiche un code ; c'est l'appareil *déjà* connecté qui le scanne et confirme — jamais l'inverse.

Pourquoi une confirmation, jamais une reconnaissance instantanée. Si le scan suffisait à connecter, un attaquant pourrait afficher le QR de connexion d'un appareil qu'il contrôle — sur un écran, un site, une affiche — et attendre qu'une victime le scanne par réflexe avec son propre compte déjà connecté. C'est une attaque documentée contre les connexions par QR (le « quishing »). L'écran de confirmation, qui montre *quel* appareil demande la connexion avant de l'accepter, est le seul rempart : il doit toujours s'afficher, sans exception ni raccourci.

3.2 Écran — Se connecter par QR

Sur le **nouvel** appareil, non encore connecté : un code, un compte à rebours, un statut (« En attente de scan... »). Le code se régénère seul à l'expiration — l'utilisateur n'a jamais un code mort sous les yeux.

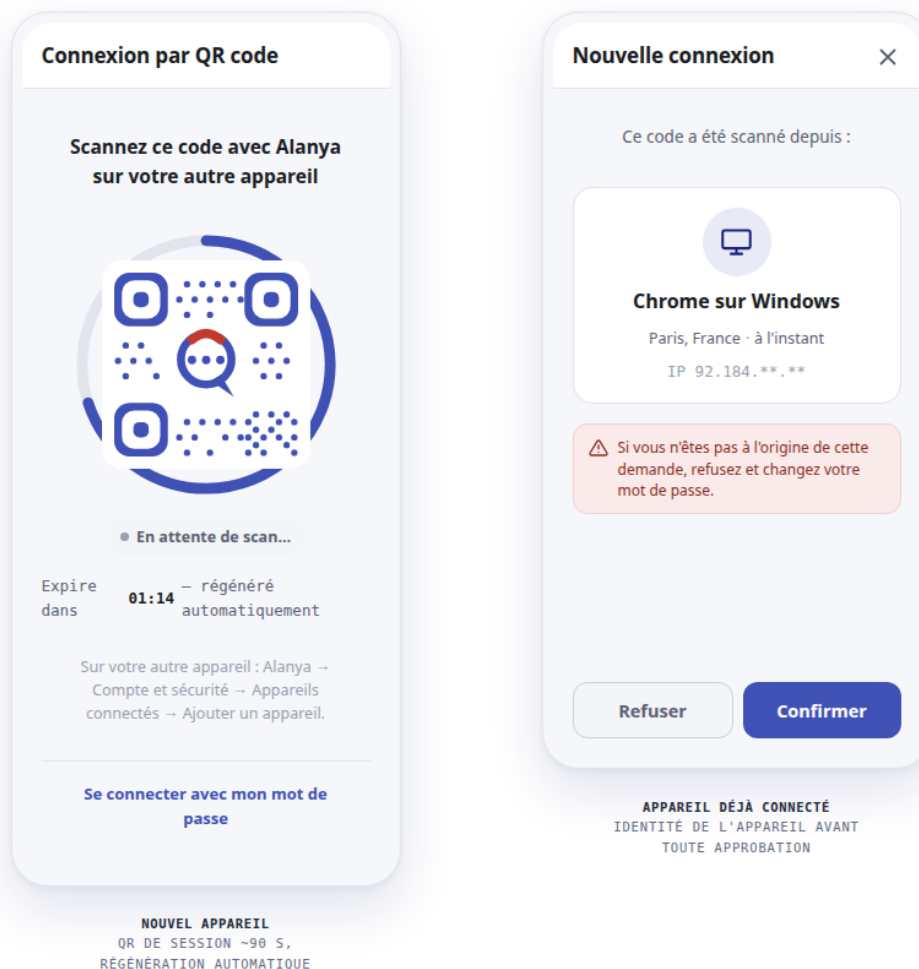
ALANYA · IDENTITÉ ET CONNEXION PAR QR CODE · ÉTAPE 6

Connecter un second appareil, jamais en un seul geste

Contrairement à l'ajout de contact, une connexion ne s'approuve jamais au simple scan. L'appareil déjà connecté doit voir qui il autorise et confirmer explicitement — le même principe que la liste des appareils, consultable à tout moment depuis le compte.

La poignée de main

Le nouvel appareil affiche, l'appareil de confiance scanne et confirme — deux écrans, un seul geste refusable.



Jamais d'approbation automatique au scan. L'écran de confirmation est obligatoire — sans lui, un QR de connexion affiché sur un site tiers et scanné par erreur suffirait à connecter un attaquant (« quishing »).

Le repli FCM ne fonctionne pas pour le nouvel appareil — il n'a pas encore d'AlanyaID à cibler. La v1 mobile n'utilise que le polling REST (~2 s) ; le canal socket reste prêt côté serveur pour un futur onglet web, qui peut rester ouvert et écouter en continu.

Maquette interactive. claude.ai/code/artifact/a69b2050

Source versionnée : docs/architecture/maquettes/qr-connexion.html

3.3 Écran — Confirmer la connexion

Sur l'appareil **déjà** connecté, après le scan : une fiche décrivant ce qui vient d'être scanné — type d'appareil, ville approximative, à l'instant — suivie de deux boutons, **Refuser** et **Confirmer**. Un avertissement rappelle de refuser et de changer son mot de passe en cas de doute.

Pourquoi l'adresse compte plus que le nom de l'appareil. Le nom et la plateforme sont *annoncés* par l'appareil qui demande la connexion : rien n'empêche un attaquant d'appeler le sien « Alanya — Vérification de sécurité ». La seule information que le serveur *constate* lui-même est l'adresse d'où part la demande. C'est donc la seule sur laquelle fonder sa décision, et l'écran le dit explicitement.

Une ville, pas une adresse technique. Cette adresse est traduite en lieu approximatif — « Douala, Cameroun » — avant d'être montrée. Sous sa forme brute (41.202.219.14), elle n'apprendrait rien à qui n'est pas technicien : on afficherait la seule information fiable de l'écran sous la seule forme où elle est inutilisable. Sous forme de ville, le doute devient une évidence — on sait immédiatement si la demande part d'où l'on se trouve. Si la traduction n'aboutit pas, l'adresse brute reste affichée plutôt que rien.

3.4 Écran — Mes appareils connectés

Un écran à part, indépendant de toute connexion en cours, atteint depuis *Compte et sécurité* : tous les appareils sur lesquels le compte est ouvert — pas seulement ceux ajoutés par QR — avec un repère « Cet appareil » et un bouton **Déconnecter** par ligne, à effet immédiat.

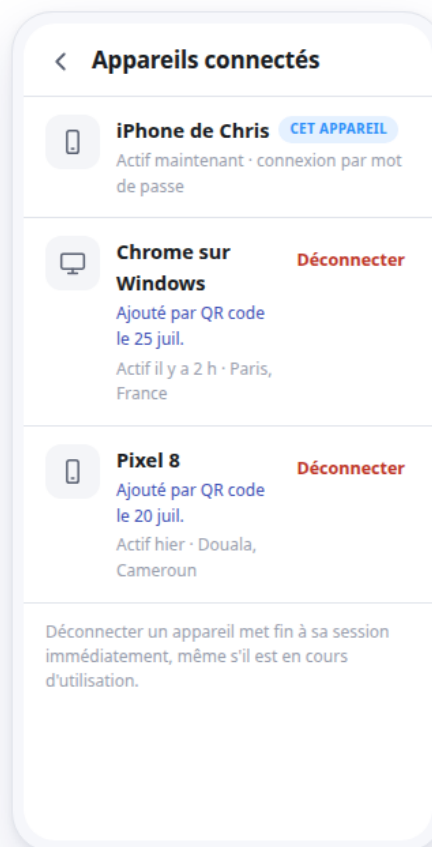
ALANYA · IDENTITÉ ET CONNEXION PAR QR CODE · ÉTAPE 6

Connecter un second appareil, jamais en un seul geste

Contrairement à l'ajout de contact, une connexion ne s'approuve jamais au simple scan. L'appareil déjà connecté doit voir qui il autorise et confirmer explicitement — le même principe que la liste des appareils, consultable à tout moment depuis le compte.

Mes appareils, à tout moment

Un écran, atteint depuis Compte et sécurité — indépendant de toute connexion QR en cours.



MES APPAREILS
TOUS LES LOGINS, PAS
SEULEMENT CEUX PAR QR

Un identifiant matériel, pas un UUID d'app. `device_id` vient de `device_info_plus` (déjà une dépendance) pour qu'un même téléphone reste le même appareil après réinstallation — pas de doublon fantôme dans la liste.

Tous les logins, pas seulement ceux par QR. Une connexion par mot de passe alimente aussi cette table — sinon l'écran resterait vide pour la quasi-totalité des comptes.

Maquette interactive. claude.ai/code/artifact/a69b2050 — second panneau de la même maquette
Source versionnée : docs/architecture/maquettes/qr-connexion.html

3.5 Points d'entrée

1. L'écran de connexion, un bouton « Se connecter avec un code QR » (`screens/authentication/login_screen.dart`), pour l'appareil qui n'est pas encore connecté.
2. *Compte et sécurité*, une nouvelle section « Appareils connectés » (`screens/profile/account_security_screen.dart`), pour lier un nouvel appareil ou consulter la liste depuis un appareil déjà connecté.

PARTIE II

Approche technique

CHAPITRE 4

Enveloppe de payload et résolution

Tous les QR d'Alanya encodent la même chose : une URL portant un **jeton opaque**, jamais une donnée brute. Ni l'`alanyaID` interne (auto-incrémenté, donc énumérable), ni l'Alanya ID (`alanyaPhone`) — ce dernier est aussi un identifiant de connexion, et l'exposer sur un support scannable par n'importe qui affaiblirait sa fonction actuelle. Trois familles de chemins, reconnues par une seule expression (`_QR_PATH_RE` dans `qrController.js`) :

Chemin encodé	Ce qu'il résout
<code>/q/c/:token</code>	un contact — code éphémère d'un compte personnel, en mémoire (section 7.1)
<code>/q/u:qrPublicId</code>	un contact — code permanent (<code>users.qr_public_id</code>), verrouillé jusqu'aux comptes business (section 5.1)
<code>/q/l:sessionId:scanSecret</code>	une session de connexion en attente, en mémoire (chapitre 8)

Une seule base d'URL, dans un seul fichier. `src/utils/qrToken.js` porte `QR_BASE_URL` (variable d'environnement, par défaut `https://www.alanya237.com`) ainsi que les trois fabricants de payload, `identityPayload`, `contactPayload` et `loginPayload`. Ils vivent ensemble et non dans leurs contrôleurs respectifs parce qu'un seul contrôleur, `qrController`, les *reparse* tous : deux constantes séparées ont déjà divergé une fois, laissant les QR de connexion sur l'ancien domaine. Le même fichier porte aussi `generateOpaqueToken` et `secretMatches`, la comparaison à temps constant des secrets. Le point qui sépare `sessionId` de `scanSecret` n'est jamais ambigu : les deux jetons sont en `base64url`, dont l'alphabet ne contient pas le point.

Les trois types partagent un seul point d'entrée :

POST <code>/api/qr/resolve</code>	Corps : <code>{ payload }</code> , la chaîne brute scannée — le client ne l'interprète jamais. Réponse : <code>{type:'contact', ...}</code> (chapitre 7) ou <code>{type:'login', session}</code> .
--	---

Le domaine n'est pas vérifié au parsing. `_parsePayload` n'extrait que le *chemin* de l'URL scannée : l'autorité de résolution est le jeton, pas l'hôte. Un QR déjà imprimé sous un ancien domaine continue donc de fonctionner, et un QR forgé sous un autre domaine ne gagne rien — il faudrait qu'il porte un jeton valide, ce que le contrôle d'autorisation vérifie ensuite.

Le scanneur doit toujours être authentifié. Les deux flux exigent que **qui scanne** soit déjà connecté — pas seulement qui possède un compte. Scanner un contact sans être soi-même connecté n'a pas de sens (à qui l'ajouter ?) ; scanner une session de connexion sans l'être serait la faille elle-même. Le middleware `authCustom` garde donc `/api/qr/resolve` au même titre que le reste de l'API, doublé de `qrResolveLimiter` (30 résolutions par minute et par IP) — une défense en profondeur contre l'énumération des jetons, pas une contrainte fonctionnelle.

CHAPITRE 5

Modèle de données

5.1 Le jeton d'identité permanent, et son verrou

Une colonne sur `users`, indépendante de `alanyaPhone` : régénérable sans effet sur la connexion, sans capacité de connexion elle-même. Elle est posée par la migration 026 ci-dessous, mais **réservée aux futurs comptes business** : le garde `_qrPermanentAutorise` (`qrController.js`) refuse aujourd'hui tout le monde. **GET** `/api/qr/me` et **POST** `/api/qr/me/regenerate` répondent 403 BUSINESS_ONLY ; la résolution d'un `/q/u/` et sa page publique répondent « inconnu ou expiré » — y compris pour les `qr_public_id` déjà renseignés par des comptes d'essai antérieurs au verrouillage. Les comptes personnels vivent sur les jetons éphémères de la section 7.1.

users.type_compte n'est pas un type de compte. Le réflexe serait de brancher ce verrou sur `users.type_compte`. Ce champ est en réalité un **rôle d'administration** — 0 utilisateur, 1 admin, 2 super-admin, posé par `PUT /users/:id/role` dans `src/routes/admin.js` — pas un type de compte métier : s'y brancher ouvrirait les codes permanents... aux administrateurs. Rien en base ne sait encore dire « ce compte est un commerce » ; c'est le volet 1 (socle de compte, `account_type`) qui l'apportera, et `_qrPermanentAutorise` s'y branchera alors — le TODO est posé dans le code, à côté du garde.

5.2 La nature de `device_id`

Un identifiant matériel, pas un UUID d'application. `lib/core/services/storage_service.dart`, méthode `getOrCreateDeviceId()`, génère un UUID aléatoire persisté par `flutter_secure_storage` — stable tant que l'app reste installée, mais **réinitialisé à la réinstallation**. Un appareil réinstallé apparaîtrait alors comme un second appareil fantôme dans la liste. Le `device_id` de la table `appareils` est donc un identifiant **matériel**, calculé par `TalkyApiClient._currentHardwareId()` et distinct de l'UUID applicatif, que le push et le socket continuent d'utiliser de leur côté.

Plateforme	Source de <code>device_id</code>
Android	<code>Settings.Secure.ANDROID_ID</code> , lu par un canal natif Kotlin — <code>MethodChannel com.alanya/device_identity</code> , méthode <code>getAndroidId</code> , déclaré dans <code>android/app/src/main/kotlin/com/alanya237/alanya/MainActivity.kt</code> . Unique par (appareil, signature d'app, utilisateur) et survivant à une réinstallation.
iOS	<code>identifierForVendor</code> , via <code>device_info_plus</code> .
Autres	repli direct sur l'UUID applicatif.

Pourquoi un canal natif et non `device_info_plus`. `device_info_plus` n'expose plus `ANDROID_ID` depuis sa **v9** — le paquet reste une dépendance et sert toujours au libellé lisible de l'appareil (`_currentDeviceModel()`) et à l'`identifierForVendor` d'iOS, mais il ne peut plus fournir la clé d'appareil sur Android. Le piège à éviter est `AndroidDeviceInfo.id`, qu'on prendrait volontiers pour son remplaçant : ce champ est `Build.ID`, c'est-à-dire l'**empreinte du firmware** : identique sur tous les téléphones d'un même modèle, et modifiée à chaque mise à jour système. Comme l'unicité porte sur le couple (`alanyaID`, `device_id`), deux téléphones du même modèle appartenant au *même* compte se retrouveraient sur une seule ligne — en déconnecter un déconnecterait l'autre — pendant qu'une simple mise à jour ferait réapparaître chaque téléphone comme neuf. Et un identifiant que tout possesseur du même modèle connaît n'est plus un identifiant. Strictement inutilisable comme clé d'appareil ; d'où le canal natif.

La chaîne de repli, jusqu'au bout. Si `ANDROID_ID` est indisponible (exception ou valeur vide), et hors Android/iOS (web, macOS, Windows, Linux), `_currentHardwareId()` retombe sur l'`UUID` applicatif `StorageService().getOrCreateDeviceId()` — puis, en dernier recours, sur `'INDEFINI'`. Mieux vaut un identifiant par installation qu'aucun : sans `device_id`, aucune ligne `appareils` n'est créée, donc aucun jeton n'est émis (chapitre 6) et la connexion échoue. Le prix de ce repli est la seule imperfection assumée du dispositif : sur ces plateformes, une réinstallation crée une seconde ligne.

Asymétrie de persistance entre plateformes. Le Keychain iOS — et donc potentiellement l'`UUID` applicatif lui-même — survit généralement à une réinstallation ; les données d'application Android, dont l'`UUID` applicatif, sont effacées à la désinstallation. `ANDROID_ID` est donc le seul des deux identifiants qui apporte une vraie amélioration côté Android. Côté iOS, `identifierForVendor` ne survit pas à la désinstallation de la *dernière* application du vendeur : le téléphone réapparaît alors comme un nouvel appareil. Limite assumée, et signalée à l'utilisateur dans l'écran « Appareils connectés ».

Conséquence assumée : `appareils.device_id` et `user_push_devices.deviceId` ne sont **pas** directement joignables — ce sont deux identifiants différents. La table `appareils` porte donc un `push_device_id` nullable, réservé à l'affichage optionnel du statut « notifications activées » dans la liste des appareils. **Aucun code ne l'écrit à ce jour** : la colonne est posée pour cet usage futur, et l'écran doit se comporter comme si elle était toujours vide.

5.3 Le jeton et la table `appareils`

```

1  -- Identite QR (users.qr_public_id) + registre des appareils connectes
2  -- (appareils), alimente par toute connexion (login, register, QR).
3  --
4  -- MySQL 8 ne supporte pas ADD COLUMN IF NOT EXISTS (cf. migration
5  -- 008) : si
6  -- relancee apres un premier passage reussi, ignorer les erreurs 1060
7  -- (Duplicate column name) / 1061 (Duplicate key name).
8
9  ALTER TABLE users
10     ADD COLUMN qr_public_id VARCHAR(64) NULL,
11     ADD UNIQUE KEY uq_users_qr_public_id (qr_public_id);
12
13 CREATE TABLE IF NOT EXISTS appareils (
14     id BIGINT AUTO_INCREMENT PRIMARY KEY,
15     alanyaID INT NOT NULL,
16     device_id VARCHAR(128) NOT NULL,
17     push_device_id VARCHAR(128) NULL,
18     device_name VARCHAR(120) NULL,
19     -- VARCHAR et non ENUM : le client envoie aussi macOS / Windows /
20     Linux, et

```

```

19  -- ajouter une plateforme ne doit pas demander un ALTER TABLE.
20  platform VARCHAR(20) NOT NULL DEFAULT 'unknown',
21  ip_address VARCHAR(64) NULL,
22  login_method ENUM('password','register','qr') NOT NULL,
23  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
24  last_active_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
25  revoked_at DATETIME NULL,
26  UNIQUE KEY uq_appareils_user_device (alanyaID, device_id),
27  KEY idx_appareils_user (alanyaID, revoked_at),
28  CONSTRAINT fk_appareils_user FOREIGN KEY (alanyaID)
29  REFERENCES users(alanyaID) ON DELETE CASCADE
30 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

```

Listing 5.1 – migrations/026_qr_appareils.sql

Ordre de déploiement — la migration d’abord, le code ensuite. Les trois gardes d’authentification (`src/middleware/auth.js`, `src/middleware/authCustom.js`, `src/socket/handlers/auth.js`) joignent appareils à **chaque** requête authentifiée. Déployer le code sans la table, c’est un `ER_NO_SUCH_TABLE` sur toute l’API — messages, appels et `/auth/me` compris, pas seulement le QR. Le dépôt n’a pas de *runner* de migrations : elles s’appliquent à la main, c’est donc à vérifier explicitement au déploiement. L’avertissement figure en tête du fichier SQL.

Pourquoi ni `user_push_devices` ni `userAccess`. Deux autres tables évoquent déjà « appareil », chacune pour une raison différente. `user_push_devices` (migration 020) a une portée strictement push — heartbeat, `appState`, jeton FCM : des écritures fréquentes et peu sensibles. `userAccess` est un journal en écriture seule, jamais muté, consultable seulement par un admin. Aucune des deux n’est un état de session vivant et révocable. Mélanger `revoked_at` — un champ sensible, rarement écrit — avec le flux à haute fréquence de `user_push_devices` aurait été le genre de raccourci qui coûte cher plus tard.

platform en `VARCHAR(20)`, login_method en `ENUM`. Les deux colonnes sont un vocabulaire fermé ; une seule est déclarée comme tel, et l’écart est délibéré. `platform` est **ouvert dans le temps** : le client envoie déjà `android`, `ios`, `web`, `macos`, `windows`, `linux` et `unknown`, et une plateforme de plus ne doit pas coûter un `ALTER TABLE` sur une table de session — d’où le `VARCHAR(20)`, sur le modèle de `user_push_devices.platform`. `login_method`, lui, est **fermé par nature** : on ne se connecte que par mot de passe, par inscription ou par QR, et un quatrième moyen serait de toute façon un chantier, pas une valeur de plus. La validation n’est donc pas laissée à la seule base : `deviceSessionService` normalise `platform` contre une liste blanche (`_normalizePlatform`, repli sur `'unknown'`) et lève sur un `login_method` inconnu, avant l’insertion.

5.4 L’origine d’un contact

```

1  -- Origine d'un contact prefere -- 'search' (recherche manuelle) ou
   'qr'
2  -- (scan ou lien) -- et note contextuelle optionnelle saisie juste
   apres un
3  -- scan. Alimentent la pastille, les filtres << Par QR >> et la fiche.
4
5  ALTER TABLE preferredContact
6  ADD COLUMN added_via VARCHAR(16) NOT NULL DEFAULT 'search',
7  ADD COLUMN added_note VARCHAR(200) NULL;

```

Listing 5.2 – migrations/027_pref_contact_added_via.sql

VARCHAR et non ENUM, même leçon que `appareils.platform` ci-dessus : le vocabulaire est ouvert dans le temps (un partage de fiche, un import de répertoire ajouteraient chacun leur origine), et une valeur de plus ne doit pas coûter un ALTER TABLE. Le DEFAULT 'search' requalifie d'un coup tout l'existant — exact, puisque la recherche était jusqu'ici le seul chemin d'ajout.

`added_note` est la note de contexte, posée *après* l'ajout par **PUT** `/api/contacts/:id/note` (corps {note}, 200 caractères, vide = effacement, 404 si la personne n'est pas un contact préféré). Elle vit sur la ligne de la relation — `alanyaID` → `idFriend` — et n'est donc jamais visible de l'intéressé ; GET `/contacts` la restitue en `addedNote`, le client Flutter la reflète en local (`local_users.preferredNote`, schéma v17) et la fiche l'affiche en italique sous la mention d'origine.

Migration 027 avant le code, comme la 026. `contactService` *nomme* la colonne dans son INSERT et ses SELECT : déployer le code sans la migration casse l'ajout de contact — REST comme QR. Même discipline que pour la 026 : appliquer d'abord, déployer ensuite. L'avertissement figure en tête du fichier SQL.

Pas de table pour la session éphémère. La session de connexion (le pont entre le nouvel appareil et celui qui l'autorise) ne vit **pas** dans une table SQL — voir la justification au chapitre 8, une fois son mécanisme introduit. Le jeton de contact éphémère (section 7.1) suit le même régime.

CHAPITRE 6

Rendre la révocation réelle

Le point de départ, avant ce volet. L'authentification était **100 % sans état** : un jeton d'accès (15 min) et un jeton de rafraîchissement (30 jours, secret distinct), tous deux de simples JWT signés. Aucune table **sessions** ni **tokens** n'existait, et `POST /api/auth/refresh` (`authCustomController.js`) se contentait de vérifier la signature pour réémettre une nouvelle paire — sans jamais rien invalider. Un jeton de rafraîchissement qui fuyait restait valide 30 jours, quoi qu'il arrive. Ce chapitre est la réponse à ce constat : la même route refuse désormais de renouveler quoi que ce soit si l'`appareilId` du jeton désigne une ligne révoquée.

Chiffré. Sur 78 comptes en production, **69** ont encore `users.device_ID = 'INDEFINI'` (la valeur par défaut de la colonne) et seulement **9** portent un vrai identifiant — hérité non pas du login, mais d'un effet de bord de `pushDevicesController.js::registerPushDevice`, qui écrase par rétrocompatibilité l'ancienne colonne mono-appareil à chaque enregistrement push réussi. Ce chemin ne couvre ni les comptes qui refusent les notifications, ni le moment précis de la connexion. La table `appareils` ne peut donc pas être un sous-produit d'un autre flux : elle doit être alimentée **directement** par `register`, `login` et l'approbation d'une session QR.

Table dédiée, pas une extension de `user_push_devices`. Réutiliser `user_push_devices` pour porter aussi la session d'authentification a été envisagé, puis écarté : les deux concepts ont des cycles de vie incompatibles sur une même ligne. Un enregistrement push doit pouvoir survivre à la révocation d'une session — c'est précisément ce qui permet de notifier un appareil qu'il vient d'être déconnecté. Les fusionner aurait rendu cette notification impossible au moment même où elle est la plus utile.

Le JWT est étendu d'un champ `appareilId` :

```
1 const generateAccessToken = (payload) =>
2   jwt.sign({ ...payload, type: 'access' }, JWT_SECRET, { expiresIn:
3     '15m' });
4   // payload devient { alanyaID, email, appareilId }
```

Listing 6.1 – `src/middleware/authCustom.js` — extension du payload

La vérification de `appareils.revoked_at IS NULL` se fait dans les gardes d'authentification, à **chaque** requête — pas seulement au rafraîchissement. Elles sont **trois**, toutes sous `src/middleware/authCustom.js`, `src/middleware/auth.js` et `src/socket/handlers/auth.js`. Oublier la troisième laisserait un appareil révoqué en 401 sur tout le REST mais parfaitement vivant en temps réel : il continuerait de recevoir les messages du compte.

```
1 SELECT u.alanyaID, u.alanyaPhone, u.email
2 FROM users u
3 LEFT JOIN appareils a ON a.id = ? AND a.alanyaID = u.alanyaID
4 WHERE u.alanyaID = ? AND u.exclus = 0
5 AND (a.id IS NULL OR a.revoked_at IS NULL);
```

Listing 6.2 – La jointure commune aux trois gardes

Pourquoi une jointure *externe*, et ce qu'elle concède. Le `LEFT JOIN` et la clause `a.id IS NULL` laissent passer les jetons émis **avant** la migration 026, qui ne portent pas d'`appareilId` — sans quoi le déploiement déconnecterait tout le monde d'un coup. La contrepartie est explicite : la révocation n'est réelle que pour les sessions ouvertes *après* la migration. Les anciennes s'éteignent d'elles-mêmes à l'expiration de leur jeton de rafraîchissement, au plus tard 30 jours après.

Le coût assumé. Cela ajoute un `SELECT` indexé (`idx_appareils_user`) sur le chemin le plus chaud de l'application — en pratique, une jointure de plus sur une requête déjà faite. Alternative plus légère : ne vérifier qu'au rafraîchissement, avec un délai de grâce allant jusqu'à 15 minutes. Choix retenu : vérifier à chaque requête. Un bouton « Déconnecter » qui met jusqu'à un quart d'heure à faire effet se comporterait, du point de vue de l'utilisateur, comme un bouton cassé.

6.1 Ce que fait vraiment la révocation

`DELETE /api/auth/device-sessions/:id` pose `revoked_at = NOW()`, puis enchaîne deux gestes de nature très différente :

1. **Un signal de confort** — l'événement `auth:device_revoked` part vers *tout* le compte, dans la room `user_${alanyaID}`, avec l'**identifiant matériel** de l'appareil coupé. Chaque client compare cette valeur à son propre `currentHardwareId()` : seul l'appareil visé se reconnaît, et il efface alors sa session locale — stockage sécurisé compris — au lieu d'attendre son prochain 401. Diffuser à tout le compte plutôt que cibler une socket est volontaire : l'appareil révoqué peut très bien ne pas avoir de socket ouverte à cet instant, et son identifiant matériel ne dit rien d'exploitable aux autres appareils du *même* compte.
2. **La garantie dure** — le 401 des trois gardes. Elle ne dépend d'aucune coopération du client. `disconnectAppareilSockets` (`src/utils/userSocketRegistry.js`) ferme en outre les sockets encore ouvertes de cet appareil, *après* l'`emit` pour que l'événement parte avant la fermeture.

La clé de fermeture des sockets est `appareilId`, pas `device_id`. `disconnectAppareilSockets` filtre sur `socket.appareilId` — l'identifiant de *ligne*, porté par le JWT et posé sur la socket à son authentification. Filtrer sur un `socket.deviceId` ne marcherait pas : cette propriété-là porte l'**UUID applicatif** du push, qui n'a aucun rapport avec `appareils.device_id` (section 5.2). Les deux champs se ressemblent assez pour qu'on les confonde, et l'erreur serait silencieuse : aucune socket ne correspondrait, aucune exception ne serait levée, et la fermeture ne ferait simplement rien.

Un appareil révoqué ne se *réactive* jamais. Quand un appareil précédemment révoqué se reconnecte, `deviceSessionService.recordLogin` ne remet pas `revoked_at` à `NULL` : il **supprime** la ligne morte et en insère une neuve. La raison tient à l'`id`. Réactiver la ligne conserverait son identifiant, donc l'`appareilId` porté par les anciens jetons : le jeton de rafraîchissement de 30 jours qu'on venait précisément de couper redeviendrait valide, et l'appareil volé retrouverait l'accès sans rien faire d'autre qu'attendre une reconnexion légitime du propriétaire. Un `id` neuf ne figure dans aucun jeton déjà émis.

Pas de ligne `appareils`, pas de jeton. `register` et `login` exigent un identifiant d'appareil (`hardware_id`, à défaut `device_ID`) et refusent la valeur sentinelle `'INDEFINI'` : sans lui, c'est un 400. Et si `recordLogin` échoue malgré tout, ils répondent **503** sans émettre le moindre jeton, plutôt que de laisser passer une connexion. Une session sans ligne `appareils` serait en effet invisible dans « Appareils connectés » et irrévocable — exactement ce que ce chapitre entier cherche à éliminer. Mieux vaut une connexion qui échoue franchement qu'une session fantôme. Le même refus s'applique à l'approbation d'une session QR.

CHAPITRE 7

Ajout de contact instantané

`addPreferredContact` (`src/controllers/preferredContactController.js`) faisait déjà exactement ce qu'il faut : rejet si l'utilisateur se scanne lui-même, vérification que la cible existe et n'est pas exclue, vérification de non-blocage, vérification de non-appartenance déjà acquise, puis INSERT. Cette logique est extraite en service — `src/services/contactService.js`, fonction `addContactByFriendId(alanyaID, friendID, {addedVia})` — partagé par le contrôleur REST existant **et** par le résolveur QR, sans dupliquer les règles. L'option `addedVia` ('search' par défaut, 'qr' pour les deux chemins QR) alimente la colonne de la migration 027, et `_contactBody` restitue `addedVia` et `addedAt` dans chaque réponse — la ligne telle qu'elle s'affichera. Le même fichier exporte `sanitizeUrl`, le filtre des sentinelles historiques de `users.avatar_url` ('NON DEFINI', 'null'...), pour la même raison : recopié, il diverge et l'avatar s'affiche différemment d'un écran à l'autre.

L'origine restituée est celle de l'ajout initial. Sur `already`, le service relit `added_via` et `created_at` de la ligne *existante* : rescanner quelqu'un qu'on avait ajouté par recherche ne réécrit pas l'histoire, et la mention datée de la fiche reste celle du premier geste.

Le service ne connaît aucun code HTTP. `addContactByFriendId` renvoie un résultat discriminé — `{ok, reason, contact}` avec `reason` ∈ `added, self, not_found, blocked, already` — et laisse chaque appelant choisir sa sémantique. C'est indispensable ici, parce que les deux appelants *divergent volontairement* sur deux cas : là où la route REST POST `/api/contacts/:id` répond 400 (soi-même) et 409 (déjà contact), la résolution d'un QR répond 200 dans les deux cas. Un service qui aurait choisi les codes à leur place aurait forcé le QR à mentir sur ce qui s'est passé.

7.1 Le jeton éphémère, en mémoire

Le code que l'écran « Mon code » affiche pour un compte personnel est un jeton de **dix minutes**, à **usage unique**, qui ne touche jamais la base : nouveau fichier `src/socket/state/qrContactTokens.js`, calque assumé de `qrLoginSessions.js` (chapitre 8, où le choix « une Map en mémoire, pas une table » est justifié — mono-processus compris).

Structure	Map token → {token, alanyaID, createdAt, expiresAt}, jeton <code>generateOpaqueToken(16)</code>
Durée de vie	TTL_MS = 10 min, renvoyée au client en <code>ttlMs</code>
Un seul jeton actif	index alanyaID → jeton ; <code>create()</code> invalide le précédent du même compte
Usage unique	<code>consume()</code> = lecture + suppression ; <code>get()</code> reste non destructif
Nettoyage	expiration paresseuse, balayage d'un échantillon à chaque création (<code>SWEEP_PER_CREATE = 20</code>), éviction FIFO au-delà de <code>MAX_TOKENS = 10 000</code>

La création est autolimitante. Un seul jeton vivant par utilisateur : en générer un nouveau tue le précédent. Marteler `POST /api/qr/contact-token` ne fait donc jamais grossir la `Map` — c’est pourquoi la route, déjà authentifiée, n’a pas de limiteur dédié. Le balayage et le plafond restent, même filet que `qrLoginSessions` : une `Map` bornée, pas seulement expirante.

Testé avec la session de connexion. `qrContactTokens.test.js` (création, invalidation du précédent, consommation unique, expiration paresseuse) est chaîné à `qrLoginSessions.test.js` dans le script `npm run test:qr`.

7.2 Résolution et consommation

`_resolveContactEphemere` (`qrController.js`) lit le jeton *sans* le consommer, applique `addContactByFriendId` avec `{addedVia: 'qr'}`, puis décide du sort du jeton selon l’issue :

Réponse de <code>POST /api/qr/resolve</code>	Cas — sort du jeton
<code>{type: 'contact', added: true, alreadyContact: false, contact}</code>	ajouté — jeton consommé
<code>{type: 'contact', added: false, alreadyContact: true, contact}</code>	déjà favori — succès doux, aucune écriture, jeton consommé quand même
<code>200 {type: 'contact', self: true}</code>	code correspondant à soi-même — le jeton survit
<code>403</code>	bloqué, dans un sens ou dans l’autre — le jeton survit
<code>404</code>	jeton inconnu, expiré, déjà utilisé, ou compte exclu

Qui a le droit de tuer un code. La règle produit « scanné = expiré » est lue littéralement : le jeton est consommé dès qu’il a rempli son office — contact ajouté, *ou* déjà présent (le scan a bien « réussi » du point de vue de qui scanne). Il survit en revanche à un scan de son propre code et à un scan par un compte bloqué : ni une maladresse du propriétaire, ni un bloqué, ne doivent pouvoir tuer le code d’autrui. Et la consommation vient **après** l’écriture, jamais avant : consommer d’abord laisserait un `INSERT` en échec détruire un code encore valide, pour rien.

Se scanner soi-même est un 200, pas un 400. La partie I range ce cas parmi les **succès doux** : « rien ne se passe, un message discret l’indique ». Une erreur 400 obligerait le client à distinguer, dans une même branche d’échec, le code illisible (400 lui aussi) de son propre code — et le moindre raccourci d’affichage transformerait un geste anodin en message d’erreur rouge. Le serveur répond donc 200 avec un drapeau `self`, que le client lit explicitement (`isSelf` dans `lib/models/qr_models.dart`). Même logique pour un contact déjà enregistré : la carte de résultat s’affiche, sans nouvelle écriture. Le champ s’appelle bien `contact` et non `user` : c’est la ligne `preferredContact` telle qu’elle apparaîtra dans la liste, `idPrefContact`, `addedVia` et `addedAt` compris, pas la fiche de l’utilisateur.

La page publique ne consomme jamais. `GET /q/c/:token` (`src/controllers/qrLandingController.js`, monté hors `/api` par `src/routes/qrLanding.js`) sert la page qu’un appareil photo générique ouvre en scannant le code : identité du propriétaire, bouton « Ouvrir dans Alanya » en schéma applicatif, mention « valable une dizaine de minutes et pour une seule personne ». Elle n’est pas authentifiée, ne sait pas qui la consulte, et **ne consomme pas** le jeton : seule la résolution par un utilisateur connecté dans l’app le fait. La page du code permanent (`GET /q/u/:token`) suit le verrou business : tant qu’il est fermé, elle répond « Ce code n’est plus valide ».

7.3 Prévenir le propriétaire

Après consommation, `_prevenirProprietaire` avertit le compte dont le code vient d’être utilisé — c’est ce qui alimente le dialogue « ajouter en retour » de la partie I. Deux canaux, jamais les deux à la fois :

1. l’événement socket `qr:contact_scanned`, vers la room `user_${alanyaID}` du propriétaire : `{by: {alanyaID, nom, pseudo, avatar_url}, alreadyMutual, at}`. `alreadyMutual` dit si le propriétaire a *déjà* le scanneur en contact — le client affiche alors une simple information au lieu du dialogue oui/non ;
2. la notification push `qr_contact_scanned` (`notificationService.notifyQrContactScanned`), **seulement** si `hasForegroundSocket` (`src/utils/userSocketRegistry.js`) ne trouve aucune socket du compte déclarée au premier plan : app visible, le dialogue in-app suffit et la push ferait doublon ; app en arrière-plan ou fermée, la push prend le relais.

Le tout est *best-effort* : l’appel n’est pas attendu et ses erreurs sont journalisées sans jamais faire échouer l’ajout — le scanneur a son contact, prévenir le propriétaire est un bonus, pas une condition.

7.4 L’ajout en retour

Accepter le dialogue rejoue simplement la route existante, **POST** `/api/contacts/:id`, avec un corps `{addedVia: 'qr'}`. Le contrôleur n’accepte que cette valeur en **liste blanche** — tout le reste retombe sur `'search'` : cette métadonnée est cosmétique (pastille, filtre, mention datée), pas de la sécurité, mais on ne laisse pas le client inventer des origines. Les deux directions du lien portent ainsi la même origine, et **GET** `/api/contacts` restitue `addedVia` et `addedAt` pour toute la liste.

CHAPITRE 8

La session de connexion éphémère

8.1 Une structure en mémoire, pas une table

Vérifié dans le déploiement. Alanya-Backend tourne en **un seul processus Node** — `npm start` lance `node server.js`, le déploiement fait `pm2 restart alanya-backend` sans mode cluster, et aucun `ecosystem.config.js` n'existe. Un état partagé uniquement en mémoire est donc visible de toutes les requêtes, sans risque qu'une requête tombe sur un autre processus qui ignore la session.

La session de connexion vit dans une `Map` en mémoire, sur le modèle exact de `src/socket/state/pendingCalls.js` : `TTL_MS`, `createdAt` / `expiresAt`, et un nettoyage porté par la structure elle-même plutôt que par un job externe. Nouveau fichier : `src/socket/state/qrLoginSessions.js` — dont le jeton de contact éphémère (section 7.1) est le calque.

Ce que ça évite. Ni migration, ni colonne, ni politique de rétention à trancher (une ligne de table qui traînerait des semaines n'existe plus, par construction — l'entrée disparaît de la `Map` à l'expiration). Ce que ça coûte, en échange : une session en cours ne survit pas à un redémarrage du serveur — l'utilisateur relance simplement le QR, un cas déjà géré par la régénération automatique ci-dessous.

Une `Map` bornée, pas seulement expirante. L'expiration est *paresseuse* : pas de `setInterval`, une entrée périmée disparaît à la première lecture qui la rencontre. Cela suffit tant qu'on lit ; or la route de création n'est pas authentifiée, et son `authLimiter` a `skipSuccessfulRequests` : les créations *réussies* ne consomment aucun quota. Un client qui créerait sans jamais interroger ferait donc croître la `Map` indéfiniment, dans le processus qui héberge aussi tous les sockets. Chaque création balaie donc un échantillon d'entrées (`SWEEP_PER_CREATE`) et évince en FIFO au-delà de `MAX_SESSIONS` — une `Map` JavaScript itère dans l'ordre d'insertion, l'éviction est donc gratuite.

8.2 Deux secrets par session, jamais interchangeables

C'est la pièce la moins évidente du dispositif, et celle dont dépend tout le reste : **le QR ne suffit pas à récupérer les jetons.**

Secret	Dans le QR ?	À qui, et pour quoi
<code>scanSecret</code>	oui	À l'appareil qui scanne : il prouve qu'on a vu le code. Exigé par <code>/resolve</code> , <code>/approve</code> et <code>/deny</code> .
<code>pollToken</code>	jamais	À l'appareil qui a créé la session, et à lui seul — remis une unique fois, dans la réponse à la création. Exigé pour interroger le statut et retirer les jetons.

Sans cette séparation, photographier le QR suffirait. Avec un secret unique, quiconque prend en photo l'écran du nouvel appareil pourrait interroger la session en boucle et **récupérer les jetons à la place du demandeur**, à l'instant précis où l'utilisateur approuve — l'écran de confirmation, lui, aurait montré un appareil parfaitement légitime. Le `pollToken` coupe cette attaque à la racine : il n'a jamais été affiché nulle part.

En-tête X-Poll-Token, et non paramètre d'URL. GET `/api/auth/qr-session/:sessionId` lit le `pollToken` dans un **en-tête** HTTP. Une *query string* serait fonctionnellement identique et discrètement désastreuse : nginx et Cloudflare journalisent l'URL complète par défaut, et la réponse à cette requête précise transporte `accessToken` et `refreshToken`. Le `sessionId` étant public — il est dans le QR — le `pollToken` est le *seul* facteur : il ne doit pas finir en clair dans un journal d'accès.

Livraison des jetons à usage unique. Dès que l'interrogation renvoie `approved` avec la paire de jetons, l'entrée est **effacée** de la Map (`clear`). Les jetons ne repassent jamais par ce canal : un second appel, avec le même `pollToken`, ne trouve plus rien. Une session inconnue et un `pollToken` faux donnent d'ailleurs la même réponse — `{status:'expired'}` — pour qu'on ne puisse pas, avec le seul `sessionId` public, sonder l'existence d'une session vivante. Même principe côté scan : un `scanSecret` invalide et une session inconnue renvoient tous deux 404.

8.3 Cycle de vie

Statut	Déclencheur
pending	créée par le nouvel appareil, <code>expiresAt</code> = maintenant + 90 s
scanned	un appareil authentifié a résolu le jeton
approving	réserve interne pendant l'approbation, jamais exposée
approved	confirmation explicite — crée la ligne <code>appareils</code> , puis livre les jetons une seule fois
denied	refus explicite. L'entrée est conservée jusqu'à son TTL : la supprimer afficherait « expiré » au demandeur, un message trompeur
expired	statut <i>synthétique</i> — l'entrée n'est plus dans la Map, qu'elle ait expiré ou déjà livré ses jetons

À expiration, le nouvel appareil régénère seul une nouvelle session — l'utilisateur ne voit jamais un code mort.

Réserve atomique de l'approbation. L'approbation n'est pas instantanée : elle enchaîne plusieurs `await` — lecture du compte, écriture dans `appareils`, signature des jetons. Deux confirmations quasi simultanées franchiraient donc toutes deux la garde de statut, et la **seconde écraserait le résultat de la première** : l'appareil demandeur ouvrirait le compte du second approbateur. `qrLoginSessions.beginApproval()` bascule le statut en `approving` *avant* le premier `await` ; comme rien n'est asynchrone entre la lecture et l'écriture du statut, la transition est atomique en JavaScript mono-thread. La seconde confirmation reçoit alors un 409. `abortApproval()` rend la session à son statut précédent si l'approbation échoue en cours de route — compte introuvable, `recordLogin` en échec — pour qu'un incident ne condamne pas un code encore valide.

Le cas où la session expire pendant l'approbation. `approve()` peut échouer après que la ligne `appareils` a été créée. Le contrôleur teste donc son retour : en cas d'échec, il **supprime la ligne** qu'il vient d'insérer et répond 410 `QR_SESSION_EXPIRED`. Sans ce test, on annoncerait « approuvé », on émettrait une paire de jetons que personne ne viendrait chercher, et on laisserait un appareil orphelin dans la liste « Appareils connectés ».

8.4 L'exception d'authentification socket

Constat vérifié dans le code. Chaque handler socket existant — chat, appels, réunions, accusés de réception — ouvre par la même garde : `if (!socket.authenticated) return;`. C'est la règle universelle du fichier `server.js` et de `src/socket/handlers/`.

Le nouvel appareil n'est, par définition, pas encore authentifié quand il attend une approbation. Un nouveau handler `src/socket/handlers/qrLogin.js` déroge donc — seule exception du dépôt — à cette garde : `socket.on('qr_session:join', ...)` valide directement contre la Map de `qrLoginSessions.js` plutôt que contre une session utilisateur, puis rejoint la room `qr_session_${sessionId}`.

Une exception bornée par construction. Elle exige le `pollToken` (section 8.2), pas le `scanSecret` : photographier le QR ne suffit donc pas à entrer dans la room. Le socket ne devient jamais `authenticated`, ne rejoint jamais `user_*` et ne reçoit donc aucun autre événement du serveur. La room ne transporte que le *statut*, jamais de jeton — la livraison des jetons reste sur le *poll* HTTP à usage unique. Et la session s'auto-détruit au bout de 90 secondes. Quatre bornes, aucune reposant sur la bonne volonté du client.

Le repli FCM ne fonctionne pas ici. `notificationService.js::sendToUser` cible par `alanyaID` — une donnée que le nouvel appareil n'a, par construction, pas encore. Le repli habituel (push en complément du socket, comme pour les appels) est donc **impossible** sur ce chemin précis. Le seul canal qui fonctionne avant authentification est celui que l'appareil a lui-même ouvert : la room socket, ou le polling REST.

Décision v1 : polling seul côté mobile. Le socket authentifié de `TalkyApiClient` suppose systématiquement un jeton d'accès valide — toute la mécanique de reconnexion et de *watchdog* est bâtie autour d'une session déjà établie. Faire cohabiter un mode « anonyme, scellé à un jeton de 90 s » dans cette même classe est un risque de régression sur un composant déjà complexe. **Le client mobile v1 n'utilise que le polling REST** (`GET /api/auth/qr-session/:sessionId`, toutes les 2 s) — le délai perçu est négligeable face au temps humain de déverrouiller l'autre téléphone et de confirmer. La room `qr_session_${sessionId}` reste implémentée côté serveur : elle ne coûte rien et prépare la voie web (chapitre 10.6), qui peut se permettre un socket ouvert en continu.

ttlMs plutôt qu'une simple date de fin. La création renvoie `expiresAt` et `ttlMs`. Le client décompte à partir de `ttlMs` et de l'instant de *réception*, sans jamais comparer son horloge à celle du serveur. Un téléphone dont l'horloge avance de plus de 90 secondes — ce qui n'a rien d'exotique — verrait sinon son code expiré dès son affichage, et le régénérerait en boucle sans jamais laisser le temps de le scanner.

Confirmation explicite, jamais d'approbation automatique. Rappel technique du choix produit : `POST /api/qr/resolve` sur un jeton de type `login` ne fait que passer le statut à `scanned` et renvoyer les informations de l'appareil demandeur — il est *strictement en lecture seule* côté autorisation. Seul `POST /api/auth/qr-session/:sessionId/approve`, appelé après une action explicite de l'utilisateur sur un écran dédié, crée la ligne `appareils` et émet les jetons. Sans cette séparation en deux appels, le scan seul suffirait à connecter — exactement le scénario de « quishing » que la partie I met en garde.

Les informations affichées viennent d'un appelant non authentifié. `deviceName` et `platform` sont fournis par l'appareil qui *crée* la session — lequel n'est, par construction, pas encore connecté. Ils s'affichent pourtant tels quels sur l'écran de confirmation, seul rempart contre le quishing. La plateforme est donc contrainte à un vocabulaire fermé **dès la création** (et non au moment de l'insertion en base, trop tard car après l'approbation), et le libellé libre est borné à 60 caractères puis présenté côté client comme « déclaré par l'appareil », jamais comme un fait vérifié.

CHAPITRE 9

API

Trois routeurs : `src/routes/qr.js`, monté sur `/api/qr` par `server.js`; `src/routes/authCustom.js`, qui accueille les chemins `/api/auth/qr-session*` et `/api/auth/device-sessions*` — exactement comme il accueille déjà `push-devices` et `notification-prefs`; et `src/routes/qrLanding.js`, les pages **publiques** servies à la racine du domaine (`/q/c/`, `/q/u/`, section 7.3) ainsi que les deux fichiers d'association des liens universels, `/.well-known/assetlinks.json` et `/.well-known/apple-app-site-association` (404 tant que l'empreinte de signature Android, resp. le Team ID Apple, ne sont pas fournis en variables d'environnement — le lien ouvre alors le navigateur, dégradation et non panne).

Endpoint	Garde	Effet
POST <code>/api/qr/contact-token</code>	<code>authCustom</code>	jeton éphémère du compte; renvoie <code>{token, payload, ttlMs, expiresAt}</code> et invalide le précédent
GET <code>/api/qr/me</code>	<code>authCustom</code>	<i>mint-if-absent</i> de <code>qr_public_id</code> ; 403 <code>BUSINESS_ONLY</code> tant que le verrou est fermé
POST <code>/api/qr/me/regenerate</code>	<code>authCustom</code>	nouveau jeton permanent, l'ancien devient invalide; même 403 que ci-dessus
POST <code>/api/qr/resolve</code>	<code>authCustom</code> + <code>qrResolveLimiter</code>	résout un jeton scanné (contact éphémère, permanent ou login)
GET <code>/q/c/:token</code> , <code>/q/u/:token</code>	aucune (public)	pages d'accueil des codes contact — jamais de consommation ni d'écriture
POST <code>/api/auth/qr-session</code>	<code>authLimiter</code> (public)	crée une session; renvoie <code>scanSecret</code> , <code>pollToken</code> , <code>payload</code> , <code>expiresAt</code> , <code>ttlMs</code>
GET <code>/api/auth/qr-session/:sessionId</code>	en-tête <code>X-Poll-Token</code> (public)	interrogation par le nouvel appareil; livre les jetons une seule fois
POST <code>/api/auth/qr-session/:sessionId/approve</code>	<code>authCustom</code> + <code>scanSecret</code>	crée appareils, émet les jetons
POST <code>/api/auth/qr-session/:sessionId/deny</code>	<code>authCustom</code> + <code>scanSecret</code>	marque <code>denied</code>
GET <code>/api/auth/device-sessions</code>	<code>authCustom</code>	liste des appareils non révoqués
DELETE <code>/api/auth/device-sessions/:id</code>	<code>authCustom</code>	révocation immédiate

Module	Rôle
src/controllers/qrController.js	/api/qr/* — jeton éphémère, verrou du permanent, résolution, notification du propriétaire
src/controllers/qrLandingController.js	pages publiques /q/c/ et /q/u/
src/controllers/qrAuthController.js	/api/auth/qr-session* et /api/auth/device-sessions* : un seul contrôleur, parce que les deux familles manipulent le même objet — la session d'un appareil — l'une pour l'ouvrir, l'autre pour la fermer
src/services/deviceSessionService.js	recordLogin / touchLastActive / normalizePlatform — la table appareils, partagée avec login et register
src/services/contactService.js	addContactByFriendId (option addedVia), partagé avec POST /api/contacts/:id
src/utils/qrToken.js	jetons opaques, payloads, comparaison à temps constant
src/socket/state/qrLoginSessions.js	la Map des sessions de connexion éphémères
src/socket/state/qrContactTokens.js	la Map des jetons de contact éphémères, un seul actif par compte

9.1 Limitation de débit

Route	Limiteur	Pourquoi ce calibre
POST /api/qr/contact-token	aucun limiteur dédié	Route authentifiée et création autolimitante : un seul jeton actif par compte, chaque création invalide le précédent (section 7.1).
POST /api/auth/qr-session	authLimiter (10 / 15 min / IP)	Création, <i>une</i> par code affiché. Le limiteur a skipSuccessfulRequests : seuls les échecs consomment le quota.
POST /api/qr/resolve	qrResolveLimiter (30 / min / IP)	Un humain qui scanne dépasse rarement quelques codes par minute.
GET /api/auth/qr-session/:sessionId	un limiteur dédié , de l'ordre de 60 / min / IP	Interrogation toutes les 2 s pendant 90 s : environ 45 appels <i>par code affiché</i> .

Ne jamais poser authLimiter sur l'interrogation de statut. Le réflexe serait d'aligner l'interrogation sur la création, toutes deux publiques. Il casserait la fonctionnalité : **authLimiter** autorise 10 requêtes par 15 minutes, quand un seul code en consomme environ 45 en 90 secondes. L'utilisateur verrait son écran se figer au bout de vingt secondes, puis échouer — et le symptôme ressemblerait à une panne réseau, pas à un plafond. Cette route a besoin d'un limiteur *dimensionné pour du polling*, de l'ordre de 60 requêtes par minute et par IP : assez large pour deux appareils derrière un même NAT, assez étroit pour rendre coûteux le sondage massif de **sessionId**. **À ce jour, aucun limiteur dédié n'est posé sur cette route** : elle n'est couverte que par le **generalLimiter** global (300 / min / IP), qui la protège d'un abus grossier mais n'est pas calibré pour elle.

CHAPITRE 10

Client Flutter

10.1 Couche données

Fichier	Contenu
<code>lib/api/qr_api.dart</code> (<i>nouveau</i>)	Extension <code>QrApi</code> on <code>TalkyApiClient</code> , part of le client, sur le modèle de <code>auth_api.dart</code> / <code>users_api.dart</code> : <code>createContactQr</code> (le code éphémère de « Mon code »), <code>resolveQr</code> , et les dormants <code>getMyQr</code> / <code>regenerateQr</code> — 403 BUSINESS_ONLY tant que le verrou du permanent est fermé, gardés pour les comptes business.
<code>lib/api/qr_auth_api.dart</code> (<i>nouveau</i>)	Extension <code>QrAuthApi</code> : <code>createQrSession</code> , <code>pollQrSession</code> (qui pose l'en-tête <code>X-Poll-Token</code>), <code>approveQrSession</code> , <code>denyQrSession</code> , <code>listDeviceSessions</code> , <code>revokeDeviceSession</code> . Séparée de la précédente parce qu'elle vise <code>/api/auth/*</code> et qu'une partie de ses appels se fait sans jeton d'accès.
<code>lib/models/qr_models.dart</code> (<i>nouveau</i>)	<code>QrIdentity</code> , <code>QrContactCode</code> (<code>token</code> , <code>payload</code> , <code>ttl</code>), <code>QrResolveResult</code> (<code>isSelf</code> , <code>added</code> , <code>alreadyContact</code>), <code>QrContactScan</code> (<code>by</code> , <code>alreadyMutual</code>), <code>QrLoginCreated</code> , <code>QrLoginPollResult</code> , <code>AppareilSession</code> .
<code>lib/api/users_api.dart</code> (<i>modifié</i>)	<code>addContact(userId, {viaQr})</code> : l'ajout <i>en retour</i> envoie <code>{addedVia: 'qr'}</code> à la route existante.
<code>lib/providers/auth_provider.dart</code> (<i>corrigé</i>)	<code>login()</code> / <code>register()</code> transmettent désormais <code>hardwareId: await TalkyApiClient.currentHardwareId()</code> — l'identifiant matériel du chapitre 5, jusqu'ici absent malgré le paramètre déjà accepté.
<code>lib/api/socket_api.dart</code> (<i>modifié</i>)	Écoute <code>auth:device_revoked</code> , <code>auth:conflict</code> et <code>qr:contact_scanned</code> — ce dernier exposé en flux <code>qrContactScans</code> , consommé par le dialogue global <i>et</i> par l'écran « Mon code ».

Ce qui passe par Drift, et ce qui n'y passe pas. Le code personnel ne touche pas le cache : éphémère par nature, il se redemande au serveur à chaque affichage. La liste des appareils non plus : donnée de **sécurité**, un cache périmé — appareil révoqué encore affiché, appareil suspect masqué — serait activement trompeur. L'**origine** des contacts, elle, est exactement une donnée de cache : `lib/core/db/app_database.dart` passe de `schemaVersion` 15 à **16**, `local_users` gagne `addedViaQr` (booléen) et `preferredAddedAt` (date, nullable) — que des `addColumn`, le cache existant n'est pas vidé.

User.addedViaQr est nullable à dessein. Côté modèle (`lib/talky_models.dart`), `addedViaQr` est un `bool? : null` signifie « la source ne portait pas cette information » (résultat de recherche, instantané d'appel), pas « faux ». `_partialUserToCompanion` (`local_cache_repository.dart`) traduit `null` en `Value.absent()` : une écriture partielle laisse la colonne telle quelle au lieu de l'écraser. Un booléen non nullable aurait fait perdre la pastille au premier passage d'une source qui ignore la relation — silencieusement, et de façon intermittente.

10.2 Interface

Fichier	Objet
<code>screens/profile/qr_scanner_screen.dart</code> (nouveau)	Scanner plein écran (<code>mobile_scanner</code>), gabarit <code>screens/chats/camera_screen.dart</code> . Autonome : appelle <code>resolveQr</code> , branche selon <code>type</code> . Carte de résultat par-dessus la caméra restée vivante, import d'une image de la galerie (<code>analyzeImage</code>).
<code>screens/profile/qr_code_screen.dart</code> (nouveau)	« Mon code » : deux onglets, <i>Mon code</i> et <i>Scanner</i> ; code éphémère, compte à rebours, partage lien ou image, régénération.
<code>widgets/qr_scan_result_card.dart</code> (nouveau)	Carte de résultat du scanner : avatar, Message / Voir détails / Annuler, disparition différée. Sans bouton d'appel : à côté d'« Annuler », une action bruyante et irréversible invite à la faute de doigt.
<code>widgets/qr_added_sheet.dart</code> (nouveau)	La même carte, en feuille modale : confirmation d'un ajout venu d'un <i>lien</i> — l'utilisateur revient d'un navigateur et doit voir <i>qui</i> a été ajouté.
<code>widgets/qr_pin.dart</code> (nouveau)	La pastille « ajouté par QR » (section 10.4).
<code>core/services/qr_contact_flow.dart</code> (nouveau)	Flux d'ajout partagé scan / lien : écriture du cache, carte ou bandeau, annulation (<code>undoAdd</code>).
<code>core/services/qr_deep_link_service.dart</code> (nouveau)	Réception des liens <code>/q/c/</code> et <code>/q/u/</code> (section 10.6).
<code>widgets/alanya_qr_view.dart</code> (nouveau)	Rendu unique des QR — identité comme connexion.
<code>screens/profile/qr_login_confirm_screen.dart</code> (nouveau)	Fiche de l'appareil scanné, Confirmer / Refuser.
<code>screens/authentication/qr_login_screen.dart</code> (nouveau)	QR de session, compte à rebours fondé sur <code>ttlMs</code> , <code>Timer.periodic</code> de <i>polling</i> toutes les 2 s.
<code>screens/profile/connected_devices_screen.dart</code> (nouveau)	Liste appareils. Le repère « Cet appareil » vient du drapeau <code>current</code> calculé par le serveur (comparaison de <code>appareils.id</code> à <code>l'appareilId</code> du jeton), pas d'une comparaison locale.
<code>screens/profile/profile_screen.dart</code>	Icône QR dans l'en-tête.
<code>widgets/add_contact_sheet.dart</code>	Bouton « Scanner un code », pastille sur les résultats.
<code>main.dart</code>	Dialogue global « ajouter en retour », rejeu des liens reçus hors session.
<code>screens/profile/preferred_contacts_screen.dart</code>	Filtre « Tous / Par QR » avec comptes.
<code>screens/chats/contact_detail_screen.dart</code>	Mention « Ajouté par QR code · date » sur la fiche.
<code>screens/chats/chats_screen.dart</code>	Pastille dans la liste des discussions (section 10.4).
<code>screens/profile/account_security_screen.dart</code>	Sections « Appareils connectés » et « Lier un appareil ».

Rendu du QR. `qr_flutter` (`QrImageView`) permet `eyeStyle` et `dataModuleStyle` personnalisés (points arrondis, couleur `brandPrimary`), un `embeddedImage` pour le logo, et un niveau de correction d'erreur `QrErrorCorrectLevel.H` (30 % de redondance) — nécessaire pour tolérer l'occlusion du logo central sans nuire à la lecture. Le scan repose sur `mobile_scanner` (CameraX et ML Kit sur Android, AVFoundation/Vision sur iOS), retenu plutôt que `qr_code_scanner` pour son socle natif et son entretien plus actif dans l'écosystème Flutter récent.

Un seul widget de rendu, et pourquoi. `alanya_qr_view.dart` existe parce que les deux rendus avaient *déjà* divergé — yeux ronds contre carrés, bicolore contre monochrome, deux fichiers de logo différents — donnant deux identités visuelles à la même fonctionnalité. Le fond blanc en thème sombre et le niveau H y sont posés une fois pour toutes : ce ne sont pas des choix esthétiques mais des conditions de lisibilité, et les laisser à chaque écran, c'était les perdre au premier oubli.

10.3 Mon code : décompte, consommation, régénération

L'écran « Mon code » appelle `createContactQr()` à l'ouverture, puis décompte à partir du `ttlMs` serveur et de l'*instant de réception* — jamais en comparant son horloge à `expiresAt`, pour la raison déjà donnée à la section 8.4 (note « `ttlMs` plutôt qu'une simple date de fin »). À zéro, il régénère sans geste ; un compteur de génération écarte les réponses tardives d'un code déjà remplacé. Il s'abonne aussi au flux `qrContactScans` : à la consommation, le code affiché est mort par définition, l'écran en recrée un aussitôt — c'est ce qui permet d'enchaîner plusieurs personnes. Le décompte est peint *sur* la carte capturée pour le partage d'image : l'image dit elle-même qu'elle est temporaire.

Partage : le lien ou l'image, jamais un mélange. Sur Android, un partage de type `image/*` transporte bien `EXTRA_TEXT`, mais la plupart des messageries l'ignorent dès qu'il y a une pièce jointe — le lien et l'Alanya ID disparaissaient silencieusement. L'écran fait donc choisir explicitement : *Partager le lien* (texte : invitation, mention de validité, Alanya ID, payload) ou *Partager l'image* (la carte en PNG, texte joint pour les applications qui l'honorent). Aucun enregistrement en galerie : une image morte en dix minutes y finirait en code cassé — l'option reviendra avec les codes permanents business.

10.4 La pastille, rendue à un seul endroit

`lib/widgets/qr_pin.dart` dessine la pastille — coin **bas-gauche**, le bas-droit appartient au point de présence, liseré à la couleur du fond porteur — et `AppAvatar` (`lib/widgets/common/app_avatar.dart`) l'expose en option `qrBadge`. Toutes les listes passant par `AppAvatar` (contacts préférés, feuille d'ajout, nouvelle discussion, sélection de membres, partage de contact, clavier d'appel...), la marque est identique partout : la poser écran par écran, c'était la perdre au premier oubli — même argument que `alanya_qr_view.dart` ci-dessus.

La liste des discussions n'a pas de User. Ses lignes sont des conversations, pas des contacts : `chats_screen.dart` observe donc `watchPreferredContacts()` et en dérive un `Set` d'identifiants ajoutés par QR, mis à jour au fil de l'eau — une lecture Drift déjà chaude, aucune requête par ligne, et la pastille suit automatiquement un ajout ou une annulation.

10.5 Le dialogue global d'ajout en retour

L'abonnement au flux `qrContactScans` qui porte le dialogue oui/non vit dans `AuthWrapper` (`main.dart`), pas sur un écran : le code a pu être partagé par lien, et son propriétaire peut être n'importe où dans l'application quand le scan survient — un abonné visible seulement sur « Mon code »

raterait l'événement (flux *broadcast*, sans mémoire tampon). `alreadyMutual` vrai court-circuite le dialogue en simple information. Accepter appelle `addContact(..., viaQr: true)` puis met le cache à jour en écriture partielle ; un 409 (déjà ajouté entre-temps, par double scan ou depuis un autre appareil) est présenté comme un succès, pas comme une erreur.

10.6 Liens profonds

Le QR encode une URL `https` précisément pour qu'un scan hors application mène quelque part (la page publique du chapitre 7) ; les liens profonds referment la boucle en ramenant cette URL dans l'app :

1. **Android** : `intent-filter` sur `www.alanya237.com` avec `pathPrefix /q/u/` et `/q/c/` (`AndroidManifest.xml`) ;
2. **iOS** : le fichier `apple-app-site-association` servi par le backend déclare `/q/u/*` et `/q/c/*` ;
3. **schéma applicatif** `alanya://q/<genre>/<jeton>`, utilisé par le bouton « Ouvrir dans Alanya » de la page publique — il fonctionne sans empreinte de signature ni Team ID.

`QrDeepLinkService` (`app_links`) extrait du lien un couple (**genre, jeton**) — `extractIdentityToken`, genres `c` et `u` — sans jamais le résoudre lui-même : `QrContactFlow` reconstruit le payload et appelle `POST /api/qr/resolve`, exactement comme après un scan caméra. Un lien reçu hors session est mis de côté (`stashToken`) et rejoué à la connexion ; un lien de type `login` est ignoré par construction — l'approbation d'un appareil exige un scan et l'écran de confirmation, jamais un lien cliqué.

Rien dans l'API n'est spécifique au mobile : une session de connexion se crée, s'interroge et s'approuve par de simples appels REST, et la room socket `qr_session_{sessionId}` (section 8.4) fonctionne identiquement pour n'importe quel client. La colonne `appareils.platform` traite déjà `'web'` comme une valeur de premier ordre — exactement le même choix que fait `user_push_devices.platform` depuis la migration 020.

Un onglet de navigateur serait même un **meilleur** client que l'application mobile pour ce flux : il reste ouvert en continu, ce qui rend le canal socket — volontairement laissé de côté côté mobile (section 8.4) — immédiatement utile plutôt qu'à préparer pour plus tard. Seul le rendu du code changerait (un `canvas` ou un `svg` plutôt que `qr_flutter`). Aucun écran web n'est conçu dans ce volet.

CHAPITRE 11

Impact fichier par fichier

11.1 Backend — Alanya-Backend

Fichier	Nature	Objet
migrations/026_qr_appareils.sql	nouveau	qr_public_id, appareils
migrations/027_pref_contact_added_via.sql	nouveau	preferredContact.added_via
src/utils/qrToken.js	nouveau	Jetons opaques, QR_BASE_URL, les trois payloads, secretMatches
src/services/contactService.js	nouveau	addContactByFriendId (option addedVia) et sanitizeUrl, extraits d'addPreferredContact
src/services/deviceSessionService.js	nouveau	recordLogin, touchLastActive, normalizePlatform
src/socket/state/qrLoginSessions.js	nouveau	Map en mémoire des sessions de connexion, sur le modèle de pendingCalls.js
src/socket/state/qrContactTokens.js	nouveau	Map des jetons de contact éphémères (10 min, usage unique, un seul actif par compte); test qrContactTokens.test.js chaîné dans test:qr
src/controllers/qrController.js	nouveau	/api/qr/* — jeton éphémère, verrou du permanent, résolution, notification du propriétaire
src/controllers/qrLandingController.js	nouveau	Pages publiques /q/c/ et /q/u/
src/controllers/qrAuthController.js	nouveau	/api/auth/qr-session* et /api/auth/device-sessions*
src/routes/qr.js	nouveau	Monte /api/qr, dont contact-token
src/routes/qrLanding.js	nouveau	Monte /q/* et les fichiers d'association /.well-known/*
src/socket/handlers/qrLogin.js	nouveau	qr_session:join, seule exception à la garde d'authentification
server.js	modifié	Montage des routes (dont les pages publiques à la racine), enregistrement du handler socket
src/routes/authCustom.js	modifié	Routes qr-session* et device-sessions*
src/middleware/rateLimiter.js	modifié	qrResolveLimiter
src/middleware/authCustom.js	modifié	Payload JWT étendu, vérification revoked_at
src/middleware/auth.js	modifié	Idem, pour rester cohérent avec authCustom.js
src/socket/handlers/auth.js	modifié	Même vérification côté socket; socket.appareilId
src/controllers/authCustomController.js	modifié	register / login : recordLogin obligatoire, 503 sinon; refreshToken :

Trois fichiers que le dossier avait prévus et qui n'existent pas. La conception initiale annonçait `src/constants/qrSessionStatus.js`, `src/services/qrIdentityService.js` et `src/services/qrLoginService.js`. Aucun n'a été écrit, et c'est délibéré : les statuts sont des chaînes déjà exhaustivement énumérées par les transitions de `qrLoginSessions.js`, et les deux services n'auraient fait que déléguer, l'un à `qrController`, l'autre à `qrLoginSessions`. Trois indications vides valent moins qu'un contrôleur qu'on lit d'un bout à l'autre.

11.2 Mobile — Talky (Flutter)

Fichier	Nature	Objet
lib/api/qr_api.dart	nouveau	Jeton éphémère et résolution
lib/api/qr_auth_api.dart	nouveau	Sessions de connexion et appareils
lib/models/qr_models.dart	nouveau	Modèles de la couche données QR, dont <code>QrContactCode</code> et <code>QrContactScan</code>
lib/screens/profile/qr_code_screen.dart	nouveau	Mon code (compte à rebours, partage lien / image)
lib/screens/profile/qr_scanner_screen.dart	nouveau	Scanner (carte de résultat, import galerie)
lib/widgets/alanya_qr_view.dart	nouveau	Rendu unique des QR
lib/widgets/qr_scan_result_card.dart	nouveau	Carte de résultat
lib/widgets/qr_added_sheet.dart	nouveau	La même carte, en feuille modale (ajout par lien)
lib/widgets/qr_pin.dart	nouveau	Pastille « ajouté par QR »
lib/core/services/qr_contact_flow.dart	nouveau	Flux d'ajout partagé scan / lien, annulation
lib/core/services/qr_deep_link_service.dart	nouveau	Liens <code>/q/c/</code> et <code>/q/u/</code> → (genre, jeton)
lib/screens/profile/qr_login_confirm_screen.dart	nouveau	Confirmation
lib/screens/authentification/qr_login_screen.dart	nouveau	QR de session
lib/screens/profile/connected_devices_screen.dart	nouveau	Mes appareils
android/.../alanya237/alanya/MainActivity.kt	modifié	Canal natif <code>com.alanya/device_identity</code> → <code>ANDROID_ID</code>
android/.../main/AndroidManifest.xml	modifié	<code>intent-filter</code> <code>/q/u/</code> et <code>/q/c/</code>
lib/talky_api_client.dart	modifié	<code>currentHardwareId()</code> , part des deux nouvelles extensions
lib/providers/auth_provider.dart	corrigé	<code>hardwareId</code> transmis au login / register
lib/api/auth_api.dart	modifié	Champ <code>hardware_id</code> dans le corps
lib/api/users_api.dart	modifié	<code>addContact(..., viaQr:)</code>
lib/api/socket_api.dart	modifié	<code>auth:device_revoked</code> , <code>auth:conflict</code> , <code>qr:contact_scanned</code>
lib/talky_models.dart	modifié	<code>SocketEvents</code> . <code>authDeviceRevoked</code> / <code>qrContactScanned</code> , <code>AccountDeviceLogin</code> , <code>User.addedViaQr</code> / <code>preferredAddedAt</code>
lib/core/db/app_database.dart	modifié	<code>schemaVersion 15</code> → <code>16</code> : <code>addedViaQr</code> , <code>preferredAddedAt</code>
lib/core/services/local_cache_repository.dart	modifié	Écritures partielles de l'origine, <code>watchPreferredContacts</code>
lib/main.dart	modifié	Dialogue global « ajouter en

Administration — hors périmètre. Aucun écran `talky-admin` n'est nécessaire à ce volet. Une vue support « appareils connectés d'un compte » serait une extension naturelle mais n'a pas été demandée — signalée au chapitre suivant, pas conçue ici.

CHAPITRE 12

Vérification

12.1 Backend

Appliquer `026_qr_appareils.sql` et `027_pref_contact_added_via.sql` **avant** de déployer le code, redémarrer, puis :

1. `npm run test:qr` → les deux stores en mémoire passent (`qrLoginSessions`, `qrContactTokens`);
2. `POST /api/qr/contact-token` deux fois → deux jetons différents, et le *premier* ne résout plus (un seul actif par compte); `GET /api/qr/me` et `POST /regenerate` → 403 `BUSINESS_ONLY`; résoudre un `/q/u/` même valide en base → 404;
3. résoudre le jeton `/q/c/` d'un autre compte → ligne `preferredContact` avec `added_via = 'qr'`, événement `qr:contact_scanned` reçu par le propriétaire (`alreadyMutual` correct), et le **re-jouer** → 404 : le jeton est consommé. Propriétaire sans socket au premier plan → push `qr_contact_scanned`; au premier plan → pas de push;
4. scanner *son propre* jeton → 200 `{self:true}`, jamais une erreur — et le jeton **survit**; même survie après le 403 d'un compte bloqué;
5. `GET /q/c/:token` dans un navigateur → la page d'identité, et le jeton résout toujours ensuite (la page ne consomme pas); `GET /q/u/:token` → « Ce code n'est plus valide »;
6. `POST /api/contacts/:id` avec `{addedVia:'qr'}` → ligne marquée `qr`; avec `{addedVia:'pigeon'}` → ligne marquée `search` (liste blanche);
7. créer une session QR, la laisser expirer → `status = expired` au polling suivant;
8. interroger une session avec un `X-Poll-Token` erroné → `expired`, indistinguable d'une session inconnue; sans en-tête du tout → 400;
9. créer une session, la résoudre depuis un second compte, **approuver** → ligne `appareils` créée, jetons valides, `appareilId` présent dans leur charge utile. Rejouer l'interrogation → `expired` : les jetons ne sont livrés qu'une fois;
10. approuver **deux fois** la même session, quasi simultanément → la seconde reçoit 409, jamais une seconde paire de jetons;
11. `DELETE /api/auth/device-sessions/:id` sur un appareil actif → sa requête authentifiée **sui-vante** échoue immédiatement, socket fermée; le même appareil qui se reconnecte obtient une ligne au `id` **différent**.

12.2 Client

`flutter pub get` → `flutter run`.

1. Ouvrir « Mon code » → compte à rebours à 10 :00, fondé sur `ttlMs`; à zéro, un code neuf s'affiche sans geste.
2. Scanner le code d'un contact → ajout instantané, carte de résultat avec « Annuler » fonctionnel, caméra toujours vivante derrière; enchaîner un second scan sans ressortir de l'écran. Rejouer le même code → « code inconnu ou expiré ».

3. Côté propriétaire, au scan : dialogue « ajouter en retour » où qu'on soit dans l'app (accepter → pastille des deux côtés) ; app en arrière-plan → notification ; scanneur déjà en contact → simple information. L'écran « Mon code », s'il est ouvert, affiche un code neuf.
4. Vérifier la distinction : pastille sur l'avatar dans les listes *et* la liste des discussions, mention « Ajouté par QR code · date » sur la fiche, filtre « Tous / Par QR » avec les bons comptes — y compris après redémarrage de l'app (cache Drift).
5. Importer depuis la galerie une capture d'un code → même carte de résultat que par la caméra ; une image sans code et un code d'une autre app → deux messages distincts.
6. Scanner son propre code, puis un code expiré → messages distincts, caméra jamais bloquée.
7. Connecter un second appareil de bout en bout : code affiché, scan depuis le premier, fiche correcte, confirmation, jetons reçus par polling.
8. **Thème sombre** : la carte du QR personnel reste blanche.
9. Réinstaller l'app sur le même appareil physique : sur **Android**, il ne réapparaît *pas* comme un second appareil dans la liste (`ANDROID_ID` stable). Sur iOS, il réapparaît si c'était la dernière application du vendeur — limite connue.
10. Déconnecter un appareil depuis un autre : l'appareil visé se déconnecte **tout seul** à la réception d'`auth:device_revoked`, sans attendre un appel réseau. Refaire le test *application fermée* : il doit alors tomber au premier 401.
11. Régler l'horloge du téléphone avec 5 minutes d'avance, puis afficher un QR de connexion : le compte à rebours doit partir de 90 s (`ttlMs`) et non expirer immédiatement.

CHAPITRE 13

Points de vigilance et questions ouvertes

Ajout instantané : le garde-fou est devenu structurel. La règle produit — ajouter sans confirmation — garde pour seul filet côté scanneur la carte de résultat annulable. Mais le régime éphémère borne désormais l'exposition côté propriétaire : dix minutes de vie, usage unique, et un propriétaire prévenu à chaque consommation. Le scénario du code affiché en contexte public (vitrine, affiche) ne concerne plus que les codes permanents — à réexaminer au moment d'ouvrir le verrou business (section 5.1).

1. **Coût de la vérification en garde** (chapitre 6) : une jointure de plus par requête authentifiée, dans **trois** fichiers quasi dupliqués (`auth.js`, `authCustom.js`, `socket/handlers/auth.js`). Assumé ici ; leur fusion serait une occasion naturelle, mais indépendante de ce volet — et le risque, en l'état, est qu'une évolution future n'en corrige que deux sur trois.
2. **Pas de limiteur dédié sur l'interrogation de statut.** `GET /api/auth/qr-session/:sessionId` n'est couvert que par le `generalLimiter` global (300 / min / IP). Un limiteur calibré pour du *polling* (~60 / min / IP) reste à poser — en aucun cas `authLimiter`, qui casserait la fonctionnalité.
3. **`auth:conflict` a désormais un émetteur.** L'événement, qui existait côté mobile sans jamais être émis, est maintenant diffusé par `authCustomController.js` (connexion par mot de passe) et par `qrAuthController.js` (approbation d'une session QR). Son nom reste discutable : il annonce une *nouvelle connexion sur le compte*, pas un conflit. Le renommer reste ouvert, et coûterait un déploiement coordonné des deux côtés.
4. **Aucune limite** au nombre d'appareils simultanés n'est proposée. Rien dans la demande ne l'impose ; un garde-fou futur (avertissement au-delà d'un seuil, à la manière de certains services bancaires) resterait compatible avec ce schéma.
5. **`push_device_id` n'est écrit par personne.** La colonne existe, nullable, et *aucun* code ne la renseigne à ce jour — ni le client, ni le backend. Elle reste posée pour le badge « notifications activées », qui demanderait de faire remonter le `deviceId` push au moment du login. En attendant, l'écran « Appareils connectés » doit se comporter comme si elle était toujours vide (aucun badge, pas d'erreur), ce qui est le cas aujourd'hui. Rappel : un utilisateur qui refuse les notifications n'aura de toute façon jamais de correspondance dans `user_push_devices`.
6. **L'identifiant matériel n'est pas un secret.** `ANDROID_ID` est diffusé à tout le compte dans `auth:device_revoked`, et `identifierForVendor` est lisible par toute application du même vendeur. Cela reste acceptable : connaître le `device_id` d'un appareil ne permet ni de s'y substituer ni de le révoquer — ces deux gestes demandent un jeton valide du compte. Mais c'est une raison de plus pour que l'API n'expose jamais ce champ dans la liste des appareils, ce qu'elle ne fait pas (pas plus que l'adresse IP).
7. **La push « code scanné » informe, elle ne rejoue pas.** Le dialogue d'ajout en retour n'est déclenché que par l'événement `socket qr:contact_scanned`, reçu app ouverte. La notification `qr_contact_scanned` (émise seulement quand aucun socket n'est au premier plan) n'est pas mise en attente côté client : un propriétaire qui ouvre l'app *après* l'avoir balayée ne revoit pas l'invitation — il lui reste la pastille et la fiche du nouveau contact pour l'ajouter à la main. Limite assumée ; un jeu demanderait de router ce type de push jusqu'au dialogue.

8. **Le verrou business est un TODO nommé.** `_qrPermanentAutorise` refuse tout le monde tant que `account_type` (volet 1) n'existe pas (section 5.1). Toute l'infrastructure du permanent — colonne, *mint-if-absent*, régénération, page publique, extensions client dormantes — attend derrière ce seul prédicat.

Fin du dossier — étape 6.

Volet indépendant — peut se construire en parallèle des cinq premiers.